TALLER PROGRAMACION FUNCIONAL

FECHA DE ENTREGA: 11/10/2026

SAMUEL BORRAS MAHECHA
JERONIMO MARTIN
LUNA RIVERA

JORGE OSWALDO VALENZUELA

PONTIFICIA UNIVERSIDAD JAVERIANA
FACULTAD DE INGENIERIA
TEORIA DE LA COMPUTACION

BOGOTA D.C.
2026

Contenido

1. DESCRIPCIÓN DEL PROBLEMA Y OBJETIVOS DEL TALLER.....3
2. CUADRO COMPARATIVO ENTRE ENFOQUES.....4
3. ANÁLISIS DE DISEÑO: ABSTRACCIÓN, MODULARIDAD Y MANEJO DE ESTADO. .4
4. VENTAJAS Y DESVENTAJAS EVIDENCIADAS.....6
5. EVIDENCIA DE COMPILACIÓN Y EJECUCIÓN EN CONSOLA.....7
6. DECLARACIÓN DE HERRAMIENTAS DE IA GENERATIVA.....8
7. CONCLUSIONES.....8

1. DESCRIPCIÓN DEL PROBLEMA Y OBJETIVOS DEL TALLER

En esta actividad se tomó un sistema base de procesamiento de transacciones comerciales que lee un archivo CSV (*ventas.csv*) con las columnas cliente, producto adquirido, precio pagado y fecha (aaaa-mm-dd).

El código original entregado por el docente junto con el enunciado (`Main.java`, versión imperativa) resuelve la tarea con un estilo tradicional imperativo: recorre listas con ciclos `for`, acumula totales en variables modificables, busca linealmente los clientes y productos ya registrados y ordena mediante algoritmos manuales de tipo burbuja. Además, no limita las consultas a un ranking de los primeros cinco.

El objetivo fue rediseñar el programa bajo los principios del paradigma funcional en Java (nuestra versión, `Main.java`), que recibe como parámetros el archivo CSV y la fecha inicial y final del periodo, y reporta los cinco clientes con más transacciones, los cinco clientes que más dinero gastaron y los cinco productos que generaron mayores ingresos. Para ello se aplicó:

- Flujos de datos mediante la API de Java Streams, combinando operaciones intermedias (`filter`, `map`, `flatMap`, `sorted`, `limit`) con operaciones terminales (`collect`, `toList`, `sum`).
- Modelos de datos inmutables apoyados en `record` (`Venta` y `Resumen`) y colecciones inmutables (`toList()`).
- Expresiones lambda y funciones de orden superior que reciben o devuelven funciones: `enPeriodo` (devuelve un `Predicate<Venta>`), `resumirPor` (recibe un `Function<Venta,String>`), `top` (recibe un `Comparator`) y `filas` (recibe un `BiFunction`).
- Funciones puras: el reporte completo se construye como un `String` sin efectos secundarios; la lectura del archivo y la impresión en consola quedan aisladas en `leerVentasCsv` y `main`.
- Ningún identificador se reasigna y no se usa ningún ciclo `for` o `while`.
- Rankings acotados al Top 5 con `limit(5)` y validación de los parámetros (número de argumentos, formato de las fechas, fecha inicial no posterior a la final y archivo legible). El programa acepta tanto el separador `;` indicado en el enunciado como la `,` del archivo de ejemplo.

2. CUADRO COMPARATIVO ENTRE ENFOQUES

Criterio	Versión base imperativa (docente)	Nuestra versión funcional
Enfoque de resolución	**Imperativo (cómo hacerlo):** se escribe la secuencia de pasos que la máquina debe seguir para llenar listas, buscar elementos y acumular valores.	**Declarativo (qué obtener):** se especifica el resultado mediante transformaciones encadenadas sobre un flujo (Stream).
Mutabilidad del estado	**Estado mutable:** se modifican atributos y acumuladores directamente (`cantidadProductos + 1`, `totalDinero + precio`).	**Inmutabilidad:** datos modelados con `record` y colecciones inmutables (`toList()`). Ningún identificador se reasigna.
Control de iteración	**Ciclos `for` y `while`** con índices `i, j` y ordenamiento burbuja manual, de costo $O(n^2)$.	**Funciones de orden superior** sin ciclos explícitos (`filter, groupingBy, sorted, limit`); el ordenamiento usa el algoritmo de la biblioteca, O(n log n).
Nivel de abstracción	**Bajo nivel:** el programador gestiona índices y la búsqueda lineal (`buscarCliente`, `buscarProducto`) elemento a elemento.	**Alto nivel:** una sola función genérica `resumirPor` agrupa por cliente o por producto según la función clave que reciba.
Efectos secundarios	**Presentes:** los métodos de ordenamiento modifican la lista recibida (in-place) y el reporte imprime mientras calcula.	**Aislados:** cada paso devuelve una colección nueva; el reporte se construye como texto y solo `main` imprime.
Alcance del reporte	Muestra hasta 10 elementos por consulta; los productos se ordenan por unidades.	Top 5 en las tres consultas; los productos se ordenan por los ingresos que generaron, como pide el enunciado.

3. ANÁLISIS DE DISEÑO: ABSTRACCIÓN, MODULARIDAD Y MANEJO DE ESTADO

3.1. Abstracción y legibilidad

En el código imperativo la lógica de negocio queda mezclada con detalles operativos. Para agrupar las compras por cliente, el programa original usa el método auxiliar `buscarCliente`, que recorre la lista elemento por elemento buscando coincidencias de texto; con n ventas y k

clientes el costo es O(n·k). Además, existen dos métodos casi idénticos (`acumularPorCliente` y `acumularPorProducto`) que solo difieren en el campo que se usa como clave.

En la versión funcional ambas agrupaciones se resuelven con una sola función de orden superior que recibe la clave como parámetro:

```
static List<Resumen> resumirPor(List<Venta> ventas,
                                Function<Venta, String> clave) {
    return ventas.stream()
            .collect(Collectors.groupingBy(clave))
            .entrySet().stream()
            .map(grupo -> new Resumen(
                    grupo.getKey(),
                    grupo.getValue().size(),
                    grupo.getValue().stream()
                        .mapToDouble(Venta::precio).sum()))
            .toList();
}

List<Resumen> clientes  = resumirPor(ventas, Venta::cliente);
List<Resumen> productos = resumirPor(ventas, Venta::producto);
```

Esto elimina código repetitivo (*boilerplate*) y reduce los puntos donde se pueden introducir errores de lógica.

3.2. Modularidad y componibilidad

En el programa procedimental los métodos de ordenamiento están acoplados a listas que se modifican en su lugar, por lo que el orden en que se llaman altera el resultado (la misma lista se reordena varias veces). En la versión funcional el programa se compone de piezas pequeñas e independientes:

1. **Filtro de fechas:** una función que devuelve un predicado puro y reutilizable.

```
static Predicate<Venta> enPeriodo(LocalDate inicio, LocalDate fin) {
    return v -> !v.fecha().isBefore(inicio) && !v.fecha().isAfter(fin);
}
```

2. **Criterios de ordenamiento como valores:** los comparadores se definen una vez y se encadenan para desempatar (por ejemplo, dos clientes con la misma cantidad de compras se ordenan por dinero gastado y luego por nombre).

```
static final Comparator<Resumen> POR_CANTIDAD =
        Comparator.comparingLong(Resumen::cantidad).reversed()
                .thenComparing(
                        Comparator.comparingDouble(Resumen::total).reversed())
                .thenComparing(POR_NOMBRE);
```

3. **Ranking genérico Top 5:** una única función recibe la lista y el criterio; se usa para las tres consultas del reporte.

```
static <T> List<T> top(List<T> elementos, Comparator<T> criterio) {
    return elementos.stream().sorted(criterio).limit(TOP).toList();
}

top(clientes, POR_CANTIDAD);   // clientes con más transacciones
top(clientes, POR_DINERO);     // clientes que más dinero gastaron
top(productos, POR_DINERO);    // productos que generaron mayores ingresos
```

3.3. Manejo de estado y efectos secundarios

La función `generarReporte` es pura: recibe las ventas y el periodo y devuelve el reporte como texto, uniendo las secciones con `Stream.of(...).collect(Collectors.joining(...))`. Las filas de cada ranking se generan con `IntStream.range(...).mapToObj(...)` en lugar de un ciclo con contador. Gracias a esto el único efecto observable del programa es la instrucción `System.out.println(ejecutar(args))` en `main`, y cualquier parte del reporte se puede probar de forma aislada. Los errores (archivo inexistente, fechas inválidas) se representan con `Optional` en lugar de códigos de retorno numéricos como el `int` que devuelve `cargarVentas` en la versión imperativa.

4. VENTAJAS Y DESVENTAJAS EVIDENCIADAS

4.1. Versión imperativa

- **Ventajas:** es una forma de programar muy familiar para quien viene de los cursos iniciales de algoritmia; el flujo de control es explícito y permite seguir paso a paso el valor de cada variable al depurar.
- **Desventajas:** el código es más largo (307 líneas frente a 198 de la versión funcional, que además incluye validaciones y comentarios adicionales), repite lógica casi idéntica para clientes y productos, y el ordenamiento burbuja es $O(n^2)$. La presencia de estado mutable compartido impide paralelizar el procesamiento de forma segura sin sincronización.

4.2. Versión funcional

- **Ventajas:** el código es compacto y autodescriptivo: cada consulta se lee como lo que calcula. Al trabajar con datos inmutables no hay riesgo de alterar accidentalmente valores ya calculados, y las funciones puras son fáciles de probar y de reutilizar. Sobre colecciones grandes, el procesamiento se puede paralelizar cambiando `.stream()` por `.parallelStream()`.
- **Desventajas:** requiere familiarizarse con lambdas, referencias a métodos y colectores (`groupingBy`, `joining`); las trazas de error dentro de un flujo son más difíciles de leer y depurar paso a paso, y la creación de objetos intermedios genera una pequeña sobrecarga de memoria administrada por el recolector de basura.

5. EVIDENCIA DE COMPILACIÓN Y EJECUCIÓN EN CONSOLA

Se compiló el programa y se ejecutó en consola pasando como argumentos el archivo de datos y el rango de fechas solicitado (Java 17 o superior).

Compilación:

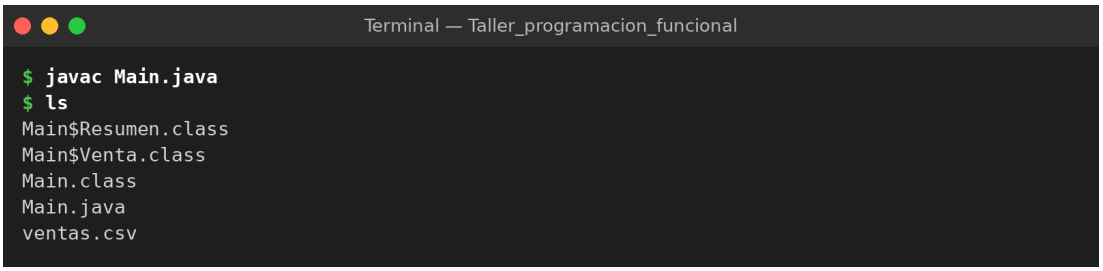


Figura 1. *Compilación con javac Main.java: no se reportan errores y se generan las clases.*

Ejecución con el periodo 2026-01-01 a 2026-06-30:

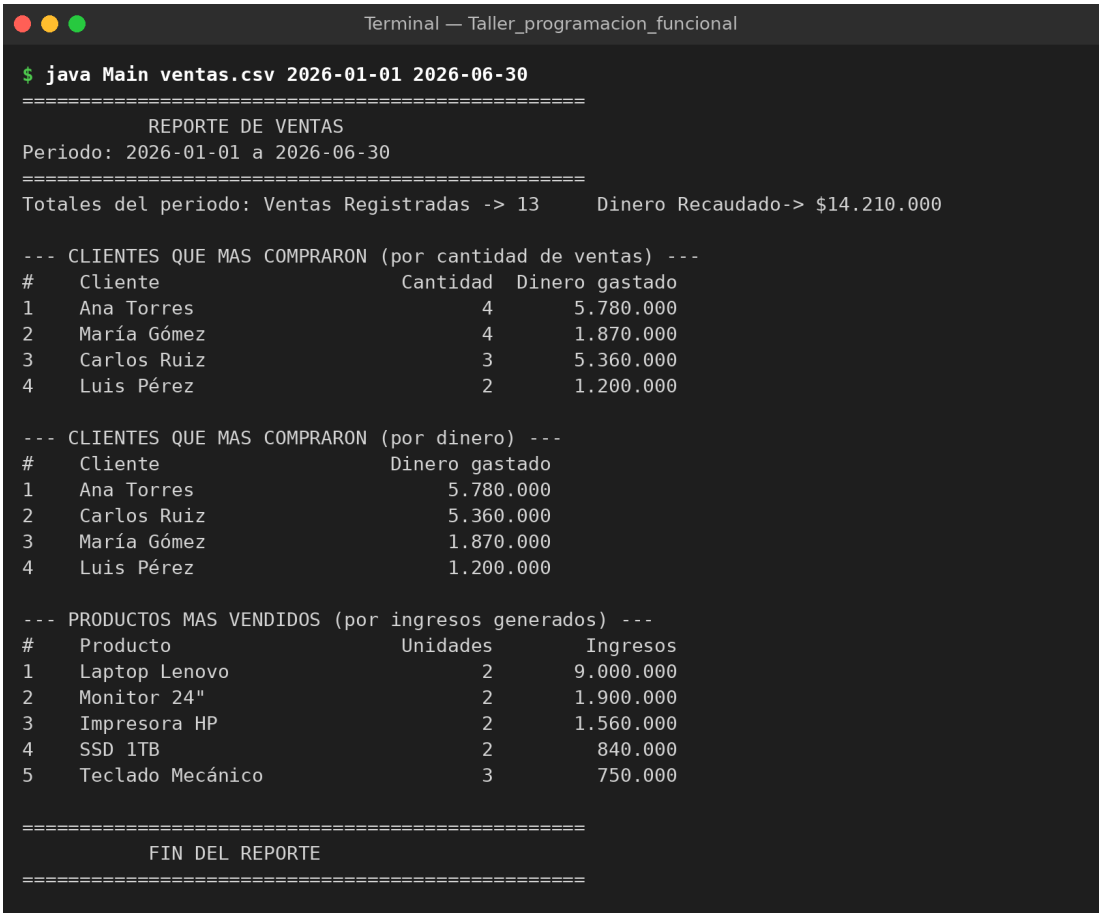


Figura 2. *Salida del programa: totales del periodo y rankings Top 5.*

El resultado reporta 13 ventas y $14.210.000 recaudados en el periodo, los mismos totales que produce el programa imperativo del docente con los mismos parámetros. En el periodo solo hay cuatro clientes con compras, por lo que los rankings de clientes muestran cuatro filas; los productos quedan ordenados por los ingresos que generaron.

Manejo de parámetros incorrectos:

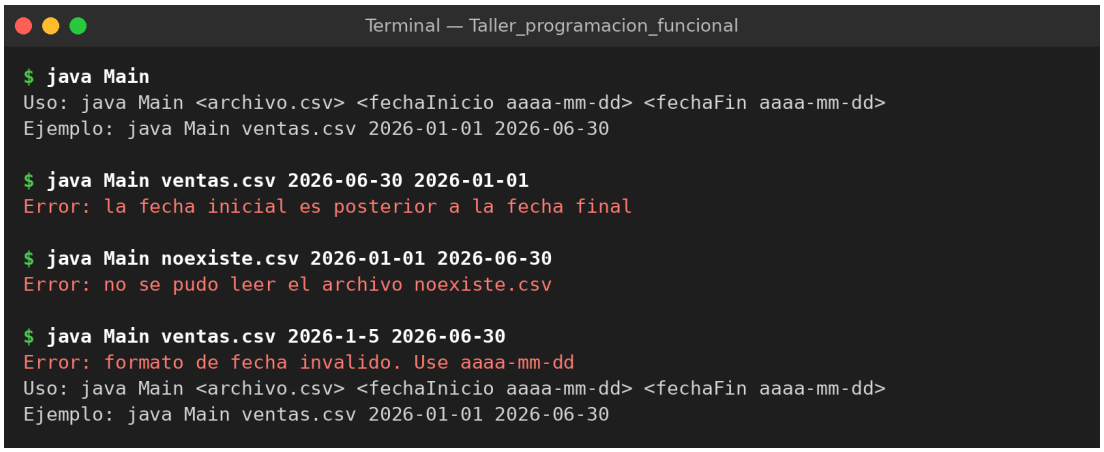


Figura 3. *Sin parámetros, fechas invertidas, archivo inexistente y fecha con formato inválido.*

6. DECLARACIÓN DE HERRAMIENTAS DE IA GENERATIVA

Conforme a las pautas académicas del curso para el uso ético y transparente de herramientas de inteligencia artificial, se detallan los usos y prompts empleados durante el desarrollo del taller:

1. “ayúdame con la lógica a implementar para que no hayan bucles en el siguiente código”.
2. Claude (Anthropic) se usó para revisar la entrega frente al enunciado y corregir los puntos encontrados: ordenar los productos por ingresos, eliminar un periodo por defecto fijo en el código, validar los parámetros, aislar la impresión en `main` y actualizar este reporte. Prompt: “revisa el zip que cumpla con todo lo requerido en el taller sino arréglalo y déjalo listo para subirlo”.

Todo el código fue revisado, compilado y ejecutado por el grupo.

7. CONCLUSIONES

1. El paradigma funcional en Java permitió reescribir la lógica del reporte de ventas reduciendo la complejidad del código: el recorrido, la acumulación y el ordenamiento se delegan a funciones de orden superior de la biblioteca estándar, y una misma función (`resumirPor`, `top`) sirve para varias consultas al recibir otra función como parámetro.
2. El uso de `record` y colecciones inmutables garantiza que los datos no cambien a lo largo del flujo, evitando errores por modificaciones colaterales como los que provoca reordenar en su lugar una lista compartida.
3. Separar las funciones puras (cálculo y formato del reporte) de los efectos (lectura del archivo e impresión) hace el programa más modular, más fácil de probar y de mantener.
4. La ejecución arrojó las mismas 13 transacciones y los mismos $14.210.000 que la versión imperativa, confirmando que la solución funcional mantiene la consistencia lógica con el programa original mientras cumple con el ranking Top 5 exigido.